

Presentation

第一页

- 1 早些年RAG还比较粗糙，一般来说用户Query通常不会很长，encoder将其编码成为dense vector。而类似的，Document会被切分成不同的chunks，每个chunk的token size可以被指定。但问题是首先尤其是对于multi-hop这种需要跨页甚至跨文档搜集相关证据的情况下表现不佳，其次实际应用下会将top-k的chunk作为evidence交给大模型去生成，即使top-k个chunk全是高相关度，由于其包含大量无关信息，可能会干扰生成质量，更不要说很多参数量不是很大的LLM本身的context length就很有有限。

You know, in the early days, RAG systems were still quite basic. User's query was usually short — the encoder just turned it into a single dense vector. Documents were split into chunks with a fixed token size, and the system picked the top-k chunks as evidence for the LLM. But that approach had clear problems. For **multi-hop reasoning** — when evidence comes from different pages or even different documents — performance dropped a lot. And even if those top-k chunks were relevant, they often carried too much noise, which hurt generation quality. On top of that, many smaller models simply don't have enough context length to handle so much text.

- 1 后来又有了以ColBERT为代表的Multi-vector Retrieval，这种是将用户Query和Document编码成token-level的多向量，为了保持token-level的细颗粒度，向量的维度是对齐的。正因为此，匹配的精准度有比较大的提升，而且这种思路很容易迁移到多模态RAG当中，即从token-level自然而然想到多模态信息间patch-level或者是pixel-level的细颗粒度对齐。

Later, we saw methods like **ColBERT**, which brought *multi-vector retrieval*. Instead of a single embedding, both queries and documents are encoded as multiple token-level vectors. This gives much finer-grained matching and works great. What's even more interesting is that this idea naturally extends to multimodal RAG — where token-level alignment becomes patch-level or pixel-level across text and images.

- 1 但问题就在于ColBERT颗粒度太细了，有的时候可能会破坏句子结构，例如Victoria Hong Kong has many what types of building? 我们当然是希望Victoria Hong Kong是作为一个整体而非分开的。这种情况下颗粒度粗一些也挺好。

But the problem is, ColBERT is *too fine-grained*. It may break sentence structure. For example: "Victoria Hong Kong has many what types of building?" Ideally, "Victoria Hong Kong" should be treated as one unit, not split apart. So in some cases, a coarser granularity actually works better.

- 1 基于上面的background，本文提出了POQD训练框架来解决这个问题，总体来说框架可分为decompose和evaluation两个部分，这也是为什么这篇论文能在ICML这种顶会发表吧。从论文题目上理解，其实就是Query Decomposer+Performance-Oriented

Based on this background, this paper proposes the **POQD training framework** to solve this issue. Overall, the framework has two parts: **decomposition** and **evaluation**. That's why it makes sense that this work got accepted at a top conference like ICML. From the title, it's clear the idea is basically *Query Decomposer + Performance-Oriented Design*.

第二页

- 1 设计思路是，将输入的prompt喂给Query Decomposer执行分解，生成的sub-queries执行MVR，最后MVR上的loss分数经由optimizer进行评估并反过来更新原始的prompt。综上所述，可以看到首先sub-queries的质量依赖prompt的质量和模型参数的更新。这一整套流程应该是一个End2end的算法。当然这里会有一个问题由于这里Query Decomposer输出的是离散的文本结构，无法将梯度直接传给下游模型的loss function，所以其实是不可微的。还有一个问题是，下游模型通常有生成器和检索器，更新并训练检索器参数通常都要重建索引并重新对语料进行嵌入，非常的花费心思，所以本篇文章采用的是只训练生成器。

The main idea of the design is this: we aim to perform query decomposition by prompting Query Decomposition with a prompt, those queries and then are then employed to perform MVR. Loss function is eventually used to evaluate and update the prompt. So, the quality of sub-queries depends on the prompt and the model parameters. In other words, the whole process works as an end2end training framework.

Of course, there are some challenges. Since the Query Decomposer outputs discrete text, we can't directly pass gradients from the downstream loss function back through it, which means it is non-differentiable. Another problem is in setups, we have both generator and retriever, updating the retriever requires rebuilding indexes and re-embedding corpus, which is extremely time-consuming. Therefore this paper focuses on training only the generator parameter instead.

- 1 上一页只是简单的阐述了项目的结构和面临的问题，接下来会详细阐述这篇论文提出的方法，方法使用两个算法交替优化，最终达到优化提示词并训练生成器至收敛的目的，并且兼顾了时间和性能的平衡。

What I just mentioned was mainly about the overall structure of the project and challenges it faces. Now let's take a closer look at the approach proposed in this paper.

The approach works through two alternating optimization algorithm, together aim to refine the prompt and train the generator until it converges. In doing so, the framework makes a balance between training efficiency and performance.

第三页

- 1 算法1的目的是，在固定下游参数的 θ 前提下，搜索一份能够让下游性能更好的提示词 p 用于驱动Query Decomposer，从而得到更优的子查询集合，损失记为 $L(\theta; p)$ 。这里只评估和生成，而非训练。

The goal of Algorithm 1 is to search for a better prompt p under the condition when downstream parameter θ is fixed. In other words, we wanna find a prompt that can drive the Query Decomposer to produce a better sub-queries, which will ultimately lead to better downstream performance. Here, the loss is defined as $L(\theta; p)$, and this section concentrates only on evaluation and generation, not on training.

- 1 模型的输入包括用于Query训练集、固定的下游模型参数和prompt，最终输出新prompt和对应的sub-queries。LS的设计很精妙，借用LLM Optimizer的思路用于描述优化趋势：通过将两端meta-prompt与LS拼接，并由Prompt Optimizer生成前缀 p_0 。整个prompt由 p_0 、task description还有query，这其实也就意味着搜索优化的 p_0 和搜索优化的p本质上是一个问题。接下来使用当前的prompt在Query Decomposer下为每一个查询生成对应的子查询，并计算损失函数。最后将 $p, L(\theta; p)$ 追加到列表当中，并返回到第一步。

The algorithm takes three inputs, the query training set, the fixed downstream model parameters, and the initial prompt. Its goal is to output an updated prompt together with a new set of sub-queries. The design of LS here is rather clever. It borrows the idea of the *LLM Optimizer* to describe the optimization trend by using natural language description. Specifically, two meta-prompts are concatenated with LS, and then the Prompt Optimizer generates a prefix called p_0 . The prompt consists of prefix p_0 , task description template and the query itself. Essentially, searching optimal p is equivalent to searching optimal prompt prefix p_0 . Next, the current prompt is used in the Query Decomposition to generate sub-queries for each input query, and the loss function is calculated. Finally, both p and its corresponding $L(\theta; p)$ are added to a list, and the process loops back to the first step

- 1 对于sub-queries不可微的问题，算法1通过不断更新LS列表，让Optimizer仅凭借这些solution-score pairs迭代改写prompt而不做梯度反传，换言之将离散结构的优化变成无梯度的搜索，从而绕开不可微的问题。

To deal with the non-differentiability of sub-queries, Algorithm 1 takes a clever method. Instead of back-propagating gradients through the discrete structure, it keeps updating the LS list, allowing the Optimizer to iteratively rewrite the prompt only based on the solution-score pairs. In other words, it turns the optimization of discrete text into a gradient-free search process. This way, it completely avoids the non-differentiable problem. d

- 1 在算法2中要开始使用POQD框架来训练了。仅靠算法1在固定的模型参数 θ 下优化分解prompt，只能带来局部的损失下降，但不能保证最终收敛后效果更好，需要将更好的prompt和下游模型训练联动起来。因此算法2的目的应该是交替的用算法1优化prompt，然后在固定该prompt的情况下训练生成器。
- 2 算法2的输入包括Query训练集、下游模型的生成器参数、随机初始化的 p^{old} ，输出是最终生成的提示词和收敛的生成器参数。

In algorithm 2, we finally start the actual training process under the POQD framework. As we mentioned before, Algorithm 1 only optimizes the decomposition prompt under fixed model θ . This helps reduce the loss locally, but it doesn't guarantee a globally convergence. To achieve this, we need to link the improved prompts with the downstream model training. Therefore, the main idea of algorithm 2 is to alternate between two steps: first recall algorithm 1 to optimize the prompt, and then with the prompt fixed, train the generator. These two processes work together iteratively until convergence. The input includes the query training set, the generator parameter of the downstream model, and a randomly initialized p^{old} . The output are the final optimized prompt and the converged generator parameters.

- 1 首先调用算法1，得到新的 p^{new} 及其优化后的子查询，当 p^{old} 等于 p^{new} 时意味着提示词无法被改进，则内层循环结束。接着固定 p^{new} ，使用优化后的sub-queries先训练模型参数 τ^{epoch} ，再将 p^{new} 赋值给 p^{old} 。最后进行收尾，固定最终的prompt，将模型训练到收敛即可。

In the first step, Algorithm 2 recalls Algorithm 1 to obtain a new prompt, along with its optimized set of sub-queries. When the old prompt becomes equal to the new prompt, it means the prompt can no longer be improved, and the inner loop stops. Next we fix the new prompt, and use optimized sub-queries to train the model parameters for τ epochs. After this training step, we update the old prompt with the current prompt and repeat the cycle.

Finally, in the last stage, we fix the final prompt and continue training the generator until full convergence. This alternating process — improving the prompt first, then training the generator, allows the POQD framework to gradually align both components and reach an optimal balance between prompt quality and model performance.

- 1 算法2采用这种小步调优(每轮少量训练)+最后全量收敛的策略非常有意思，不仅让人联想起warmup strategy。如果在每个epoch都将模型参数训练到收敛，代价就太大了，所以就采取这种交替优化+小步更新的策略。

The training strategy used in Algorithm 2 is quite interesting, follows an incremental training approach, meaning each round only trains for a few epochs before performing a full convergence at the end. This is somewhat similar to a warm-up strategy. If the model parameters were trained to full convergence at every single epoch, the cost would be extremely high. So instead, this alternating optimization with incremental updates keeps the framework efficient while still moving steadily toward a better solution.